Les Fontion et la Gestion des Erreurs en C#

Par Robert DIASSÉ

Les Fonctions en C#

1. Introduction aux Fonctions

Une fonction est un bloc de code réutilisable qui effectue une tâche spécifique. Elle permet de structurer le programme en morceaux logiques, facilitant la maintenance et la lisibilité.

2. Déclaration et Utilisation d'une Fonction Personnalisée

En C#, une fonction se déclare avec un type de retour, un nom, et éventuellement des paramètres.

Exemple 1 : Fonction sans paramètre et sans retour

```
using System;

class Program
{
    // Déclaration d'une fonction qui affiche un message
    static void AfficherMessage()
    {
        Console.WriteLine("Bienvenue dans le cours sur les fonctions en C#!");
    }

    static void Main()
    {
        // Appel de la fonction
        AfficherMessage();
    }
}
```

Explication :

- `static void AfficherMessage()` : Déclare une fonction `AfficherMessage` sans retour (`void`) et sans paramètre.

- `Console.WriteLine(...)` : Affiche un message à l'écran.
- `AfficherMessage();` : Appelle la fonction dans `Main()`.

Exemple 2 : Fonction avec paramètre(s)

```
using System;

class Program
{
    // Fonction qui prend un paramètre
    static void DireBonjour(string nom)
    {
        Console.WriteLine("Bonjour, " + nom + " !");
    }

    static void Main()
    {
        // Appel avec un argument
        DireBonjour("Alice");
    }
}
```

Explication :

- `static void DireBonjour(string nom)`: Déclare une fonction avec un paramètre `nom` de type `string`.
- `Console.WriteLine(...)`: Affiche un message incluant la valeur passée.
- `DireBonjour("Alice")`: Appelle la fonction en passant `"Alice"` comme argument.

Exemple 3 : Fonction avec retour de valeur

```
using System;

class Program
{
    // Fonction qui retourne un entier
    static int Addition(int a, int b)
    {
        return a + b;
    }

    static void Main()
    {
        int resultat = Addition(5, 3);
        Console.WriteLine("La somme est : " + resultat);
    }
}
```

Explication :

- `static int Addition(int a, int b)`: Fonction qui prend deux entiers et retourne leur somme.
- `return a + b;`: Retourne la somme de `a` et `b`.
- `int resultat = Addition(5, 3);`: Stocke le résultat de l'appel.
- `Console.WriteLine(...)`: Affiche le résultat.

---

3. Utilisation des Fonctions du Langage

C# fournit de nombreuses fonctions intégrées.
Voici une liste des fonctions usuelles en C# regroupées par catégorie, avec leur signification et leur utilisation.

---

1. Fonctions Mathématiques (`Math`)

C# offre plusieurs fonctions mathématiques intégrées dans la classe `Math`.

Fonction	Description	Exemple
`Math.Abs(x)`	Retourne la valeur absolue de `x`.	`Math.Abs(-5) → 5`
`Math.Sqrt(x)`	Calcule la racine carrée de `x`.	`Math.Sqrt(25) → 5`
`Math.Pow(x, y)`	Calcule `x` élevé à la puissance `y`.	`Math.Pow(2, 3) → 8`
`Math.Round(x)`	Arrondit `x` à l'entier le plus proche.	`Math.Round(3.6) → 4`
`Math.Floor(x)`	Arrondit `x` vers le bas (entier inférieur).	`Math.Floor(3.9) → 3`
`Math.Ceiling(x)`	Arrondit `x` vers le haut (entier supérieur).	`Math.Ceiling(3.1) → 4`
`Math.Max(x, y)`	Retourne le maximum entre `x` et `y`.	`Math.Max(4, 10) → 10`
`Math.Min(x, y)`	Retourne le minimum entre `x` et `y`.	`Math.Min(4, 10) → 4`

Exemple :

```
using System;

class Program
{
    static void Main()
    {
        double valeur = -9.5;
        Console.WriteLine("Valeur absolue : " + Math.Abs(valeur));
        Console.WriteLine("Arrondi supérieur : " + Math.Ceiling(valeur));
    }
}
```

Explication :

- `Math.Abs(-9.5) → 9.5`
- `Math.Ceiling(-9.5) → -9`

---

2. Fonctions de Manipulation de Chaînes (`String`)

C# propose de nombreuses méthodes intégrées pour manipuler les chaînes de caractères.

Fonction	Description	Exemple
`string.ToUpper()`	Convertit en majuscules.	`"hello".ToUpper() → "HELLO"`
`string.ToLower()`	Convertit en minuscules.	`"HELLO".ToLower() → "hello"`
`string.Trim()`	Supprime les espaces en début et fin.	`" hello ".Trim() → "hello"`
`string.Substring(x, y)`	Extrait `y` caractères à partir de `x`.	`"abcdef".Substring(2, 3) → "cde"`
`string.Replace("x", "y")`	Remplace `x` par `y`.	`"hello".Replace("l", "w") → "hewwo"`
`string.Contains("x")`	Vérifie si la chaîne contient `x`.	`"hello".Contains("ll") → true`
`string.IndexOf("x")`	Renvoie la position de `x`.	`"hello".IndexOf("l") → 2`
`string.Length`	Retourne la longueur de la chaîne.	`"hello".Length → 5`
`string.Split('x')`	Découpe la chaîne en tableau.	`"a,b,c".Split(',') → ["a", "b", "c"]`

Exemple :

```
using System;

class Program
{
    static void Main()
    {
        string texte = " Bonjour C# ";
        Console.WriteLine("Longueur : " + texte.Length);
        Console.WriteLine("Sans espaces : '" + texte.Trim() + "'");
    }
}
```

Explication :

- `texte.Length → 12` (compte les espaces).
- `texte.Trim() → "Bonjour C#"` (suppression des espaces en début et fin).

---

3. Fonctions de Gestion des Dates (`DateTime`)

La classe `DateTime` permet de manipuler les dates et heures.

Fonction	Description	Exemple
`DateTime.Now`	Récupère la date et l'heure actuelles.	`DateTime.Now → 2025-02-20 14:30`
`DateTime.Today`	Récupère la date du jour.	`DateTime.Today → 2025-02-20`
`DateTime.Year`	Extrait l'année.	`DateTime.Now.Year → 2025`
`DateTime.Month`	Extrait le mois.	`DateTime.Now.Month → 2`
`DateTime.Day`	Extrait le jour.	`DateTime.Now.Day → 20`
`DateTime.AddDays(x)`	Ajoute `x` jours.	`DateTime.Today.AddDays(5) → 2025-02-25`
`DateTime.ToString("format")`	Formate la date.	`DateTime.Now.ToString("dd/MM/yyyy") → "20/02/2025"`

Exemple :

```
using System;

class Program
{
    static void Main()
    {
        DateTime aujourdHui = DateTime.Now;
        Console.WriteLine("Date actuelle : " + aujourdHui.ToString("dd/MM/yyyy
HH:mm"));
        Console.WriteLine("Dans 10 jours : " +
aujourdHui.AddDays(10).ToString("dd/MM/yyyy"));
    }
}
```

Explication :

- `DateTime.Now → Date et heure actuelles.`
- `DateTime.Now.AddDays(10) → Ajoute 10 jours à la date actuelle.`

---

4. Fonctions d'Entrée/Sortie (`Console`)

Fonction	Description	Exemple
`Console.WriteLine(x)`	Affiche `x` avec retour à la ligne.	`Console.WriteLine("Hello")`
`Console.Write(x)`	Affiche `x` sans retour à la ligne.	`Console.Write("Hello")`
`Console.ReadLine()`	Lit une entrée utilisateur.	`string nom = Console.ReadLine();`
`Console.ReadKey()`	Attend un appui sur une touche.	`Console.ReadKey();`

Exemple :

```
using System;

class Program
{
    static void Main()
    {
        Console.Write("Entrez votre nom : ");
        string nom = Console.ReadLine();
        Console.WriteLine("Bonjour, " + nom + " !");
    }
}
```

Explication :

- `Console.ReadLine()` capture l'entrée utilisateur.
- `Console.WriteLine()` affiche un message avec le nom saisi.

---

5. Fonctions de Manipulation des Collections (`List<T>`, `Array`)

Fonction	Description	Exemple
`list.Add(x)`	Ajoute un élément `x`.	`list.Add(5);`
`list.Remove(x)`	Supprime `x`.	`list.Remove(5);`
`list.Count`	Retourne le nombre d'éléments.	`list.Count → 3`
`array.Length`	Retourne la taille du tableau.	`array.Length → 5`

Exemple :

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
```

```
    {
        List<int> nombres = new List<int>() { 1, 2, 3 };
        nombres.Add(4);
        nombres.Remove(2);

        Console.WriteLine("Taille : " + nombres.Count);
    }
}
```

Explication :

- `nombres.Add(4) → Ajoute 4`
- `nombres.Remove(2) → Supprime 2`
- `nombres.Count → Retourne 3`

---

4. Gestion des Erreurs dans les Fonctions

Lorsqu'on travaille avec des fonctions, des erreurs peuvent survenir. C# propose `try-catch` pour capturer et gérer les exceptions.

Exemple 5 : Gestion des erreurs avec `try-catch`

```
using System;

class Program
{
    static int Division(int a, int b)
    {
        try
        {
            return a / b;
        }
        catch (DivideByZeroException)
        {
            Console.WriteLine("Erreur : Division par zéro !");
            return 0; // Valeur par défaut
        }
    }

    static void Main()
    {
        int resultat = Division(10, 0);
        Console.WriteLine("Résultat : " + resultat);
    }
}
```

Explication :

- `try` : Contient le code susceptible de générer une erreur.
- `catch (DivideByZeroException)` : Capture l'erreur de division par zéro et affiche un message.
- `return 0;` : Fournit une valeur de secours pour éviter un crash.

---

5. Exercices Pratiques

Exercice 1 : Calculer le carré d'un nombre

Écrire une fonction qui prend un nombre en paramètre et retourne son carré.

Exercice 2 : Vérifier si un nombre est pair

Créer une fonction qui vérifie si un nombre est pair ou impair et affiche le résultat.

Exercice 3 : Trouver le plus grand entre deux nombres

Écrire une fonction qui prend deux nombres et retourne le plus grand.

Exercice 4 : Conversion Celsius en Fahrenheit

Créer une fonction qui convertit une température de Celsius en Fahrenheit.

Exercice 5 : Fonction de factorielle avec gestion d'erreurs

Écrire une fonction qui calcule la factorielle d'un nombre en gérant les erreurs d'entrée.

Conclusion :
Dans ce cours, nous avons vu comment :

- Définir et appeler une fonction en C#.
- Utiliser les fonctions intégrées du langage.
- Gérer les erreurs dans les fonctions.